

✓ PIPELINE DE LIMPIEZA DE DATOS

Cargar librerías:

```
1 from google.colab import drive
2 from sklearn.linear_model import LinearRegression
3 import pandas as pd
4 import os
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 from sklearn.decomposition import PCA
8 from sklearn.preprocessing import StandardScaler
9 import numpy as np
```

Proceso ETL - Extract, transform and load the data

› 1. Importar datos (Extract the data)

↳ 2 cells hidden

✓ 2. Transform the data

En este paso llevaremos a cabo el EDA (Exploratory Data Analysis para asegurar la limpieza y claidad de los datos)

✓ 1. Data Overview:

```
1 print("Las columnas con sus respectivos tipo de datos son: ")
2 df.info()
```

Las columnas con sus respectivos tipo de datos son:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1334 entries, 0 to 1333

Data columns (total 17 columns):

#	Column	Non-Null Count	Dtyp
0	Año	1334 non-null	int64

```

1  Cantón 1334 non-null object
2  Provincia 1334 non-null object
3  Empleo total registrado 1334 non-null float64
4  Empleo masculino registrado 1334 non-null float64
5  Empleo femenino registrado 1334 non-null float64
6  Salario promedio total registrado 1334 non-null float64
7  Salario mensual promedio masculino registrado 1334 non-null float64
8  Salario mensual promedio femenino registrado 1334 non-null float64
9  Ventas locales 1326 non-null float64
10 Recaudación tributaria 1326 non-null float64
11 Valor agregado bruto de producción 1105 non-null float64
12 Población 1105 non-null float64
13 Número de nuevas compañías 1326 non-null float64
14 Valor agregado bruto de producción per cápita 1105 non-null float64
15 variación interanual de la producción per cápita 1105 non-null float64
16 variación interanual real de la producción per cápita 1105 non-null float64
dtypes: float64(14), int64(1), object(2)
memory usage: 177.3+ KB

```

```

1 # 1. Definir las columnas a utilizar
2 cols_interes = [
3     'Año', 'Cantón', 'Provincia',
4     'Empleo total registrado',
5     'Salario promedio total registrado', 'Ventas locales', 'Recaudación tri
6     'Población', 'Número de nuevas compañías',
7     'Valor agregado bruto de producción per cápita',
8     'variación interanual real de la producción per cápita'
9 ]
10
11 # 2. Filtrar el DataFrame original
12 df_limpio = df[cols_interes].copy()

```

```

1 print("Las dimensiones del dataset usado son: ")
2 df_limpio.shape

```

Las dimensiones del dataset usado son:
(1334, 11)

✓ 2. Revisión de NA

```

1 # Cuenta los nulos por columna
2 nulos_por_columna = df_limpio.isnull().sum()
3
4 # Muestra el resultado
5 print(nulos_por_columna)

```

Año	0
Cantón	0
Provincia	0

```

Empleo total registrado      0
Salario promedio total registrado      0
Ventas locales              8
Recaudación tributaria      8
Población                  229
Número de nuevas compañías      8
Valor agregado bruto de producción per cápita      229
variación interanual real de la producción per cápita      229
dtype: int64

```

Revisión de NA por cada variable:

✓ 1. Recaudación tributaria, Ventas locales y Número de nuevas compañías

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['Recaudación tributaria'].isnull()]
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name=
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	El Piedrero	2019	1
1	El Piedrero	2020	1
2	El Piedrero	2021	1
3	El Piedrero	2022	1
4	Manga del Cura	2019	1
5	Manga del Cura	2020	1
6	Manga del Cura	2021	1
7	Manga del Cura	2022	1

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['Ventas locales'].isnull()]
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name=
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	El Piedrero	2019	1
1	El Piedrero	2020	1
2	El Piedrero	2021	1
3	El Piedrero	2022	1
4	Manga del Cura	2019	1
5	Manga del Cura	2020	1
6	Manga del Cura	2021	1
7	Manga del Cura	2022	1

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['Número de nuevas compañías'].isnull()]
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name=
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	El Piedrero	2019	1
1	El Piedrero	2020	1
2	El Piedrero	2021	1
3	El Piedrero	2022	1
4	Manga del Cura	2019	1
5	Manga del Cura	2020	1
6	Manga del Cura	2021	1
7	Manga del Cura	2022	1

Se ha determinado la eliminación de los registros correspondientes a los cantones **El Piedrero y La Manga del Cura** del dataset de entrenamiento. Esta decisión se fundamenta en los siguientes criterios técnicos:

Heterogeneidad Administrativa: Dichas zonas fueron históricamente territorios en disputa y no cuentan con la estructura administrativa de un GAD Municipal (Gobierno Autónomo Descentralizado). En consecuencia, las variables económicas críticas como Recaudación tributaria o Número de nuevas compañías presentan valores nulos estructurales, no aleatorios.

Prevención de Sesgos de Imputación: La imputación de datos económicos para estas zonas mediante promedios provinciales introduciría un "ruido" sistemático. Al ser territorios sin recaudación tributaria formal ni registros mercantiles, asignarles valores promedio de cantones formalmente establecidos sesgaría el modelo, induciendo predicciones de riesgo de pobreza artificialmente distorsionadas.

Integridad de la Muestra: Dado que estos registros representan menos del 1% del volumen total de datos, su exclusión mejora la robustez del modelo y asegura que la clasificación de riesgo de pobreza se realice sobre una base de comparabilidad institucional equitativa (cantones con competencias municipales plenas).

```

1 # 1. Definir la lista de exclusión
2 zonas_no_delimitadas = ['El Piedrero', 'Manga del Cura']
3
4 # 2. Filtrar el DataFrame principal de manera permanente
5 df_limpio = df_limpio[~df_limpio['Cantón'].isin(zonas_no_delimitadas)].copy

```

```

6
7 # 3. Eliminar explícitamente cualquier registro residual de esos cantones
8 if 'df_proyeccion' in locals():
9     df_proyeccion = df_proyeccion[~df_proyeccion['Cantón'].isin(zonas_no_
10
11 cantones_reales = df_limpio['Cantón'].unique()
12 verificacion = [c for c in zonas_no_delimitadas if c in cantones_reales]
13
14 if not verificacion:
15     print("Los cantones han sido eliminados totalmente del dataset.")
16     print(f"Nueva forma del dataset: {df_limpio.shape}")
17 else:

```

Los cantones han sido eliminados totalmente del dataset.
Nueva forma del dataset: (1326, 11)

✓ 2. Población

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['Población'].isnull()]
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name=
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	24 De Mayo	2024	1
1	Aguarico	2024	1
2	Alausí	2024	1
3	Alfredo Baquerizo Moreno (Juján)	2024	1
4	Ambato	2024	1
..	...	...	...
215	Yacuambi	2024	1
216	Yantzaza	2024	1
217	Zamora	2024	1
218	Zapotillo	2024	1
219	Zaruma	2024	1

[220 rows x 3 columns]

```

1 # 1. Crear las filas para 2024 (si no existen)
2 cantones = df_limpio['Cantón'].unique()
3 for canton in cantones:
4     # Verificamos si la fila de 2024 ya existe para e
5     if not ((df_limpio['Cantón'] == canton) & (df_lim
6         # Creamos una fila nueva con valores NaN para
7         nueva_fila = pd.DataFrame({'Cantón': [canton]
8         df_limpio = pd.concat([df_limpio, nueva_fila]
9

```

```
10 # 2. Ahora sí, calculamos la proyección e imputamos s
11 for canton in cantones:
12     df_canton = df_limpio[(df_limpio['Cantón'] == can
13
14     if len(df_canton) >= 2:
15         X = df_canton[['Año']]
16         y = df_canton['Población']
17
18         model = LinearRegression()
19         model.fit(X, y)
20
21         pob_2024 = model.predict([[2024]])[0]
22
23         # Imputamos directamente en el df_limpio
24         mask = (df_limpio['Cantón'] == canton) & (df_
25         df_limpio.loc[mask, 'Población'] = int(pob_20
26
27     print("Proyección imputada exitosamente en el DataFra
```

```
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
Proyección imputada exitosamente en el DataFrame original.  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning  
warnings.warn(  

```

1 # Ver las últimas 10 filas del dataset
2 print(df_limpio.tail(10))

1324	2021	General Antonio Elizalde	Guayas	969.583333
1325	2022	General Antonio Elizalde	Guayas	1078.333333
1326	2023	General Antonio Elizalde	Guayas	1075.000000
1327	2024	General Antonio Elizalde	Guayas	1045.000000
1328	2019	Isidro Ayora	Guayas	1407.750000
1329	2020	Isidro Ayora	Guayas	1482.750000
1330	2021	Isidro Ayora	Guayas	1617.333333
1331	2022	Isidro Ayora	Guayas	1758.583333
1332	2023	Isidro Ayora	Guayas	1893.083333
1333	2024	Isidro Ayora	Guayas	2179.250000

Salario promedio total registrado

Ventas locales \

1324	518.868242	5.747295e+07
1325	527.764867	5.702375e+07
1326	547.860097	6.416238e+07
1327	568.639069	7.972888e+07
1328	603.512375	1.833221e+08
1329	602.361740	1.987831e+08
1330	616.040424	2.428541e+08
1331	646.373169	2.769964e+08
1332	694.442134	2.861354e+08
1333	690.341644	3.043177e+08

Recaudación tributaria

Población

Número de nuevas compañías \

1324	1.405914e+06	12846.0	6.0
1325	1.678898e+06	12870.0	7.0
1326	1.862247e+06	12910.0	7.0
1327	1.651931e+06	12931.0	3.0

https://colab.research.google.com/drive/1Uvgwd42kLCHM_RBgnVE9DuitXgyNsw5e#scrollTo=3qZeQQ1_nJtz&printMode=true

7/29

```

1329      8.785173e+06      14550.0      1.0
1330      4.655559e+06      14775.0      1.0
1331      5.594993e+06      15032.0      2.0
1332      6.232149e+06      15306.0      2.0
1333      1.639944e+07      15563.0      7.0

```

```

      Valor agregado bruto de producción per cápita \
1324                                     3095.852405
1325                                     3239.243978
1326                                     3397.663052
1327                                     NaN
1328                                     2049.578068
1329                                     2180.391753
1330                                     2041.601354
1331                                     2226.173497
1332                                     2158.745590
1333                                     NaN

```

```

      variación interanual real de la producción per cápita
1324                                     3.701906
1325                                     1.167911
1326                                     2.670448
1327                                     NaN
1328                                     8.349140
1329                                     6.721068
1330                                    -6.506819
1331                                     5.576736
1332                                    -5.249041
1333                                     NaN

```

✓ 3. Valor agregado bruto de producción per cápita

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['Valor agregado bruto de producción per cápita']
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name='cantidad_nulos')
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	24 De Mayo	2024	1
1	Aguarico	2024	1
2	Alausí	2024	1
3	Alfredo Baquerizo Moreno (Juján)	2024	1
4	Ambato	2024	1
..	...	...	...
215	Yacuambi	2024	1
216	Yantzaza	2024	1
217	Zamora	2024	1
218	Zapotillo	2024	1
219	Zaruma	2024	1

[220 rows x 3 columns]

```
1  # 1. Crear las filas para 2024 (si no existen)
2  cantones = df_limpio['Cantón'].unique()
3  for canton in cantones:
4      # Verificamos si la fila de 2024 ya existe para e
5      if not ((df_limpio['Cantón'] == canton) & (df_lim
6          # Creamos una fila nueva con valores NaN para
7          nueva_fila = pd.DataFrame({'Cantón': [canton]
8          df_limpio = pd.concat([df_limpio, nueva_fila]
9
10 # 2. Ahora sí, calculamos la proyección e imputamos s
11 for canton in cantones:
12     df_canton = df_limpio[(df_limpio['Cantón'] == can
13
14     if len(df_canton) >= 2:
15         X = df_canton[['Año']]
16         y = df_canton[['Valor agregado bruto de produ
17
18         model = LinearRegression()
19         model.fit(X, y)
20
21         pob_2024 = model.predict([[2024]])[0]
22
23         # Imputamos directamente en el df_limpio
24         mask = (df_limpio['Cantón'] == canton) & (df_
25         df_limpio.loc[mask, 'Valor agregado bruto de
26
27     print("Proyección imputada exitosamente en el DataFra
```

```

/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
Proyección imputada exitosamente en el DataFrame original.
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: Userl
warnings.warn(
/tmp/ipykernel_34210/2850974798.py:25: DeprecationWarning: Conversion of an ar
df_limpio.loc[mask, 'Valor agregado bruto de producción per cápita'] = int(p

```

```

1 # Ver las últimas 10 filas del dataset
2 print(df_limpio.tail(10))

```

1324	2021	General Antonio Elizalde	Guayas	969.583333
1325	2022	General Antonio Elizalde	Guayas	1078.333333
1326	2023	General Antonio Elizalde	Guayas	1075.000000
1327	2024	General Antonio Elizalde	Guayas	1045.000000
1328	2019	Isidro Ayora	Guayas	1407.750000
1329	2020	Isidro Ayora	Guayas	1482.750000
1330	2021	Isidro Ayora	Guayas	1617.333333
1331	2022	Isidro Ayora	Guayas	1758.583333
1332	2023	Isidro Ayora	Guayas	1893.083333
1333	2024	Isidro Ayora	Guayas	2179.250000

	Salario promedio total registrado	Ventas locales \
1324	518.868242	5.747295e+07
1325	527.764867	5.702375e+07
1326	547.860097	6.416238e+07

```

1328      603.512375      1.855221e+08
1329      602.361740      1.987831e+08
1330      616.040424      2.428541e+08
1331      646.373169      2.769964e+08
1332      694.442134      2.861354e+08
1333      690.341644      3.043177e+08

      Recaudación tributaria  Población  Número de nuevas compañías \
1324      1.405914e+06      12846.0      6.0
1325      1.678898e+06      12870.0      7.0
1326      1.862247e+06      12910.0      7.0
1327      1.651931e+06      12931.0      3.0
1328      6.975473e+06      14244.0      1.0
1329      8.785173e+06      14550.0      1.0
1330      4.655559e+06      14775.0      1.0
1331      5.594993e+06      15032.0      2.0
1332      6.232149e+06      15306.0      2.0
1333      1.639944e+07      15563.0      7.0

      Valor agregado bruto de producción per cápita \
1324      3095.852405
1325      3239.243978
1326      3397.663052
1327      3463.000000
1328      2049.578068
1329      2180.391753
1330      2041.601354
1331      2226.173497
1332      2158.745590
1333      2210.000000

      variación interanual real de la producción per cápita
1324      3.701906
1325      1.167911
1326      2.670448
1327      NaN
1328      8.349140
1329      6.721068
1330      -6.506819
1331      5.576736
1332      -5.249041
1333      NaN

```

✓ 4. Variación interanual real de la producción per cápita

```

1 # 1. Filtramos las filas donde la columna específica tiene nulos
2 df_nulos = df_limpio[df_limpio['variación interanual real de la producción
3
4 # 2. Agrupamos por 'Cantón' y 'Año' y contamos
5 conteo_nulos = df_nulos.groupby(['Cantón', 'Año']).size().reset_index(name=
6
7 # 3. Mostrar el resultado
8 print(conteo_nulos)

```

	Cantón	Año	cantidad_nulos
0	24 De Mayo	2024	1
1	Aguarico	2024	1
2	Alausí	2024	1
3	Alfredo Baquerizo Moreno (Juján)	2024	1
4	Ambato	2024	1
..	...	...	...
215	Yacuambi	2024	1
216	Yantzaza	2024	1
217	Zamora	2024	1
218	Zapotillo	2024	1
219	Zaruma	2024	1

[220 rows x 3 columns]

```

1  # 1. Crear las filas para 2024 (si no existen)
2  cantones = df_limpio['Cantón'].unique()
3  for canton in cantones:
4      # Verificamos si la fila de 2024 ya existe para e
5      if not ((df_limpio['Cantón'] == canton) & (df_lim
6          # Creamos una fila nueva con valores NaN para
7          nueva_fila = pd.DataFrame({'Cantón': [canton]
8          df_limpio = pd.concat([df_limpio, nueva_fila]
9
10 # 2. Ahora sí, calculamos la proyección e imputamos s
11 for canton in cantones:
12     df_canton = df_limpio[(df_limpio['Cantón'] == can
13
14     if len(df_canton) >= 2:
15         X = df_canton[['Año']]
16         y = df_canton[['variación interanual real de
17
18         model = LinearRegression()
19         model.fit(X, y)
20
21         pob_2024 = model.predict([[2024]])[0]
22
23         # Imputamos directamente en el df_limpio
24         mask = (df_limpio['Cantón'] == canton) & (df_
25         df_limpio.loc[mask, 'variación interanual rea
26
27     print("Proyección imputada exitosamente en el DataFra

```


	0
Año	0
Cantón	0
Provincia	0
Empleo total registrado	0
Salario promedio total registrado	0
Ventas locales	0
Recaudación tributaria	0
Población	0
Número de nuevas compañías	0
Valor agregado bruto de producción per cápita	0
variación interanual real de la producción per cápita	0

dtype: int64

✓ 3. Revisión de formatos

```

1  for columna in df_limpio.columns:
2      print("\n_____")
3      print(f"Valores únicos en: {columna} ---")
4
5      num_unicos = df_limpio[columna].unique()
6      print(f"Valores únicos: {num_unicos}")

```

Valores únicos: [216440. 219063. 221282. ... 15032. 15306. 15563.]

Valores únicos en: Número de nuevas compañías ---

Valores únicos: [1.020e+02 1.960e+02 2.110e+02 2.030e+02 6.000e+00 1.100e+01 2
3.200e+01 2.600e+01 4.000e+00 1.000e+01 2.500e+01 2.900e+01 1.200e+01
1.800e+01 3.000e+01 3.300e+01 5.800e+01 5.200e+01 1.000e+00 0.000e+00
5.000e+00 2.000e+00 3.000e+00 1.300e+01 7.000e+00 4.360e+02 5.970e+02
8.270e+02 1.080e+03 1.149e+03 1.151e+03 9.000e+00 2.400e+01 1.500e+01
3.700e+01 2.100e+01 3.400e+01 1.600e+01 8.000e+00 1.700e+01 1.030e+02
1.560e+02 1.790e+02 2.430e+02 2.120e+02 2.830e+02 1.900e+01 1.400e+01
2.200e+01 3.900e+01 2.300e+01 6.700e+01 6.100e+01 9.200e+01 8.300e+01
9.500e+01 7.000e+01 6.200e+01 6.600e+01 1.190e+02 9.000e+01 2.800e+01
1.380e+02 1.300e+02 2.140e+02 1.820e+02 2.090e+02 4.200e+01 6.300e+01
4.900e+01 2.320e+02 1.990e+02 3.110e+02 3.530e+02 4.140e+02 4.020e+02
4.700e+01 2.000e+01 3.800e+01 4.600e+01 3.600e+01 3.100e+01 5.400e+01
8.100e+01 5.500e+01 6.500e+01 3.007e+03 3.489e+03 5.439e+03 6.418e+03
6.451e+03 6.206e+03 5.000e+01 7.900e+01 7.300e+01 5.600e+01 5.900e+01
1.420e+02 1.880e+02 2.210e+02 1.320e+02 1.660e+02 2.810e+02 3.280e+02
3.440e+02 3.290e+02 4.100e+01 4.400e+01 4.800e+01 4.000e+01 3.500e+01
1.130e+02 1.100e+02 6.400e+01 6.800e+01 9.300e+01 8.500e+01 5.100e+01
1.050e+02 1.270e+02 1.350e+02 1.760e+02 1.680e+02 2.500e+02 2.980e+02
3.230e+02 3.200e+02 8.700e+01 8.600e+01 9.900e+01 9.100e+01 9.700e+01
7.100e+01 5.300e+01 7.800e+01 7.400e+01 6.900e+01 1.330e+02 1.550e+02
1.670e+02 1.110e+02 1.140e+02 1.610e+02 1.720e+02 2.670e+02 2.690e+02
3.910e+02 4.230e+02 4.180e+02 4.560e+02 1.160e+02 2.852e+03 3.170e+03
4.467e+03 5.286e+03 5.172e+03 4.639e+03 1.120e+02 1.970e+02 3.120e+02
3.830e+02 3.650e+02 1.580e+02 2.240e+02 1.910e+02 8.200e+01 3.250e+02
4.870e+02 6.010e+02 5.290e+02 6.120e+02]

Valores únicos en: Valor agregado bruto de producción per cápita ---

Valores únicos: [3909.087646 3236.445041 3302.530436 ... 2226.173497 2158.7455
2210.]

Valores únicos en: variación interanual real de la producción per cápita ---

Valores únicos: [-1.57959597 -16.86855161 1.90048234 ... -6.50681894 5.5
-5 249041071

Realizamos un strip para evitar vacíos antes y después de cada palabra

```
1 # 1. Identificamos solo las columnas que son de tipo objeto
2 cols_texto = df_limpio.select_dtypes(include=['object']).columns
3
4 # 2. Aplicamos el strip a cada una de esas columnas
5 for col in cols_texto:
6     df_limpio[col] = df_limpio[col].str.strip()
7 df_limpio.head()
```

	Año	Cantón	Provincia	Empleo total registrado	Salario promedio total registrado	Ventas locales	Recaudación tributaria	P
0	2019	Ibarra	Imbabura	36354.83333	681.037298	1.306461e+09	55595874.88	
1	2020	Ibarra	Imbabura	33938.58333	680.388767	1.008828e+09	48235019.51	
2	2021	Ibarra	Imbabura	33788.50000	695.749373	1.231320e+09	57090619.35	
3	2022	Ibarra	Imbabura	35072.00000	712.406398	1.367972e+09	69732403.50	
4	2023	Ibarra	Imbabura	35493.83333	745.633942	1.462298e+09	70317766.83	

Next steps: [Generate code with df_limpio](#) [New interactive sheet](#)

Realizamos un redondeo para limpiar el formato de los datos numéricos

```
1 # 1. Identificamos columnas numéricas con decimales (float64)
2 cols_float = df_limpio.select_dtypes(include=['float64']).columns
3
4 # 2. Aplicamos el redondeo a 2 decimales
5 df_limpio[cols_float] = df_limpio[cols_float].round(2)
6
7 df_limpio.head()
```

	Año	Cantón	Provincia	Empleo total registrado	Salario promedio total registrado	Ventas locales	Recaudación tributaria	Pc
0	2019	Ibarra	Imbabura	36354.83	681.04	1.306461e+09	55595874.88	
1	2020	Ibarra	Imbabura	33938.58	680.39	1.008828e+09	48235019.51	
2	2021	Ibarra	Imbabura	33788.50	695.75	1.231320e+09	57090619.35	
3	2022	Ibarra	Imbabura	35072.00	712.41	1.367972e+09	69732403.50	
4	2023	Ibarra	Imbabura	35493.83	745.63	1.462298e+09	70317766.83	

Next steps: [Generate code with df_limpio](#) [New interactive sheet](#)

✓ 3. Analítica descriptiva

Realizamos la estadística descriptiva de las variables numéricas

```
1 df_limpio[cols_float].describe()
```

	Empleo total registrado	Salario promedio total registrado	Ventas locales	Recaudación tributaria	Población	Núm com
count	1326.000000	1326.000000	1.326000e+03	1.326000e+03	1.326000e+03	1326.
mean	12524.511750	696.587647	9.544625e+08	7.189415e+07	7.994024e+04	73.
std	70093.679715	124.253940	6.440871e+09	6.014075e+08	2.765089e+05	468.
min	93.830000	441.730000	6.004236e+05	6.436302e+04	2.193000e+03	0.
25%	706.562500	613.147500	2.097496e+07	7.658832e+05	1.259275e+04	4.
50%	1528.085000	690.075000	6.632157e+07	1.815664e+06	2.702500e+04	8.
75%	4946.230000	766.672500	2.426029e+08	8.099512e+06	6.256175e+04	22.
max	884817.830000	1480.800000	8.570419e+10	9.768836e+09	2.952313e+06	6451.

```
1 # Iteramos sobre las columnas que contienen texto
2 for col in cols_texto:
3     print(f"--- Frecuencia para la variable: {col} ---")
4
5     # Calculamos frecuencia absoluta (conteo) y porcentaje
6     frec_absoluta = df_limpio[col].value_counts()
7     frec_porcentual = df_limpio[col].value_counts(normalized=True)
8
9     # Combinamos ambas en un solo DataFrame para una mejor visualización
10    df_frecuencias = pd.DataFrame({
11        'Absoluta': frec_absoluta,
12        'Porcentual (%)': frec_porcentual.round(2)
13    })
14
15    print(df_frecuencias)
16    print("\n" + "="*40 + "\n")
```

--- Frecuencia para la variable: Cantón ---		
	Absoluta	Porcentual (%)
Cantón		
Olmedo	12	0.90
Bolívar	12	0.90
Ibarra	6	0.45
Otavaló	6	0.45
Pimampiro	6	0.45

```

...
Coronel Marcelino Maridueña      6      0.45
Lomas De Sargentillo             6      0.45
Nobol                             6      0.45
General Antonio Elizalde         6      0.45
Isidro Ayora                     6      0.45

[219 rows x 2 columns]

=====

--- Frecuencia para la variable: Provincia ---
                        Absoluta  Porcentual (%)
Provincia
Guayas                150        11.31
Manabí                 132         9.95
Loja                   96         7.24
Azuay                  90         6.79
El Oro                 84         6.33
Los Ríos               78         5.88
Morona Santiago        72         5.43
Chimborazo             60         4.52
Zamora Chinchipe       54         4.07
Tungurahua            54         4.07
Pichincha              48         3.62
Cañar                  42         3.17
Cotopaxi               42         3.17
Esmeraldas             42         3.17
Sucumbíos              42         3.17
Bolívar                42         3.17
Imbabura               36         2.71
Carchi                 36         2.71
Napo                   30         2.26
Pastaza                24         1.81
Orellana               24         1.81
Galápagos              18         1.36
Santa Elena            18         1.36
Santo Domingo de los Tsáchilas  12         0.90

=====

```

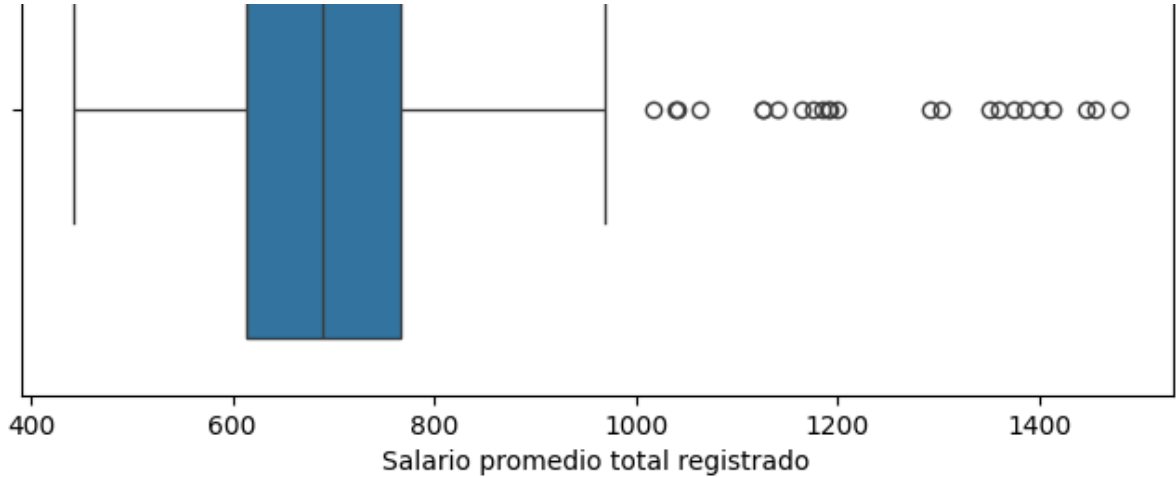
✓ 4. Graficos univariados

```

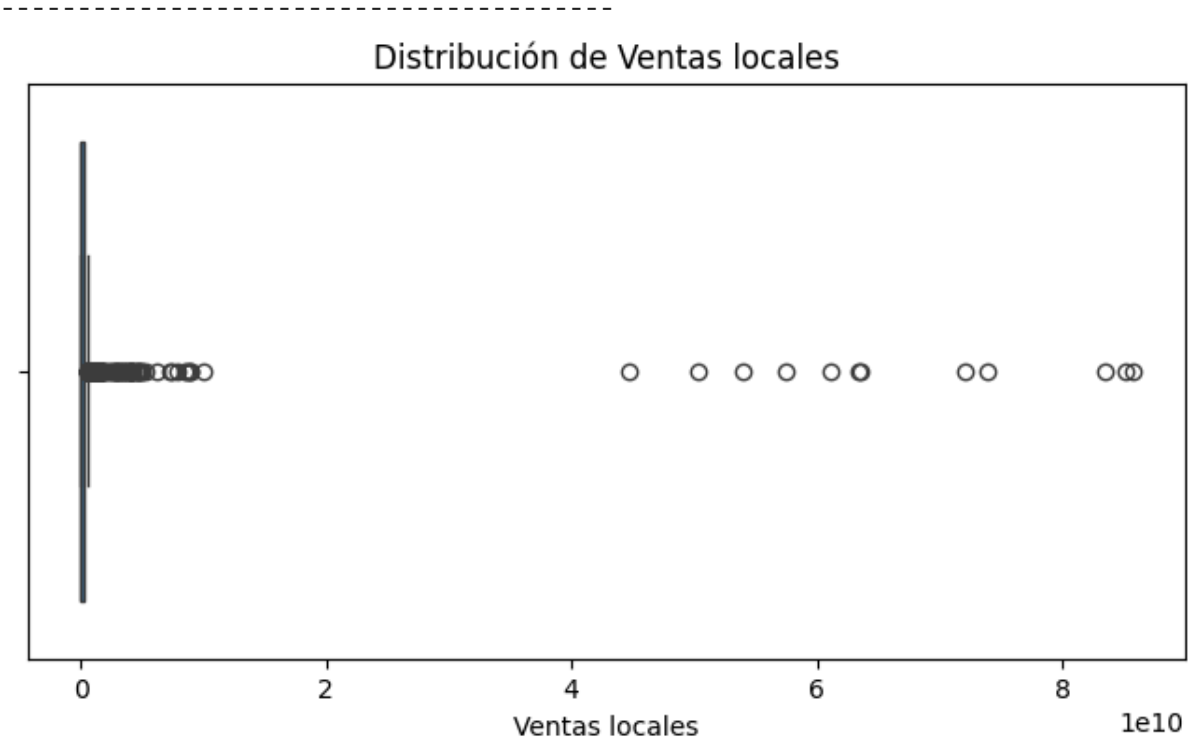
1  # 1. Identificar columnas numéricas (excluyendo el Año)
2  cols_numericas = df_limpio.select_dtypes(include=['float', 'int'])
3  cols_numericas = [c for c in cols_numericas if c != 'Año']
4
5  # Obtenemos el total de registros para el cálculo porcentual
6  total_registros = len(df_limpio)
7
8  # 2. Iterar y generar análisis
9  for col in cols_numericas:

```

```
10     # --- Cálculos Estadísticos ---
11     q1 = df_limpio[col].quantile(0.25)
12     q2 = df_limpio[col].quantile(0.50) # Mediana
13     q3 = df_limpio[col].quantile(0.75)
14     iqr = q3 - q1
15     media = df_limpio[col].mean()
16
17     # Detección de outliers (regla de 1.5 * IQR)
18     outliers = df_limpio[(df_limpio[col] < (q1 - 1.5
19     num_outliers = len(outliers)
20
21     # Cálculo del porcentaje
22     porcentaje_outliers = (num_outliers / total_regis
23
24     # --- Impresión de Resultados ---
25     print(f"--- Análisis Estadístico: {col} ---")
26     print(f"Q1: {q1:.2f} | Mediana: {q2:.2f} | Q3: {q3:.2f}")
27     print(f"Media: {media:.2f}")
28     print(f"Outliers detectados: {num_outliers} ({porcentaje_outliers:.2f}%)")
29     print("-" * 40)
30
31     # --- Gráfico ---
32     plt.figure(figsize=(8, 4))
33     sns.boxplot(x=df_limpio[col])
34     plt.title(f'Distribución de {col}')
35     plt.show()
```

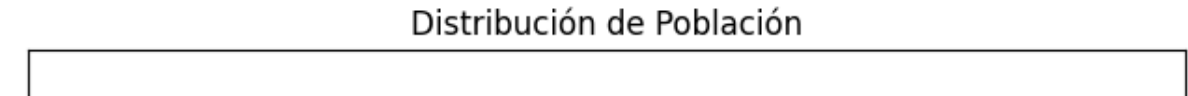
--- Análisis Estadístico: Ventas locales ---
Q1: 20974957.56 | Mediana: 66321572.35 | Q3: 242602949.28
Media: 954462524.22
Outliers detectados: 174 (13.12% del total)

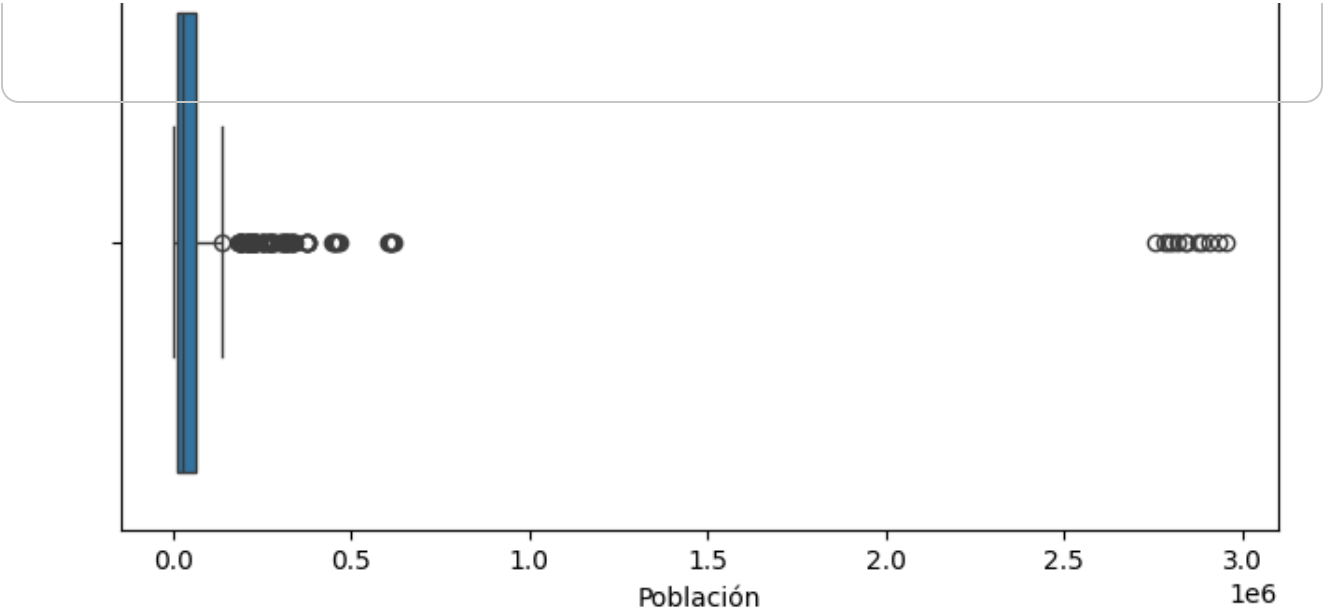


--- Análisis Estadístico: Recaudación tributaria ---
Q1: 765883.16 | Mediana: 1815663.52 | Q3: 8099512.47
Media: 71894150.40
Outliers detectados: 189 (14.25% del total)



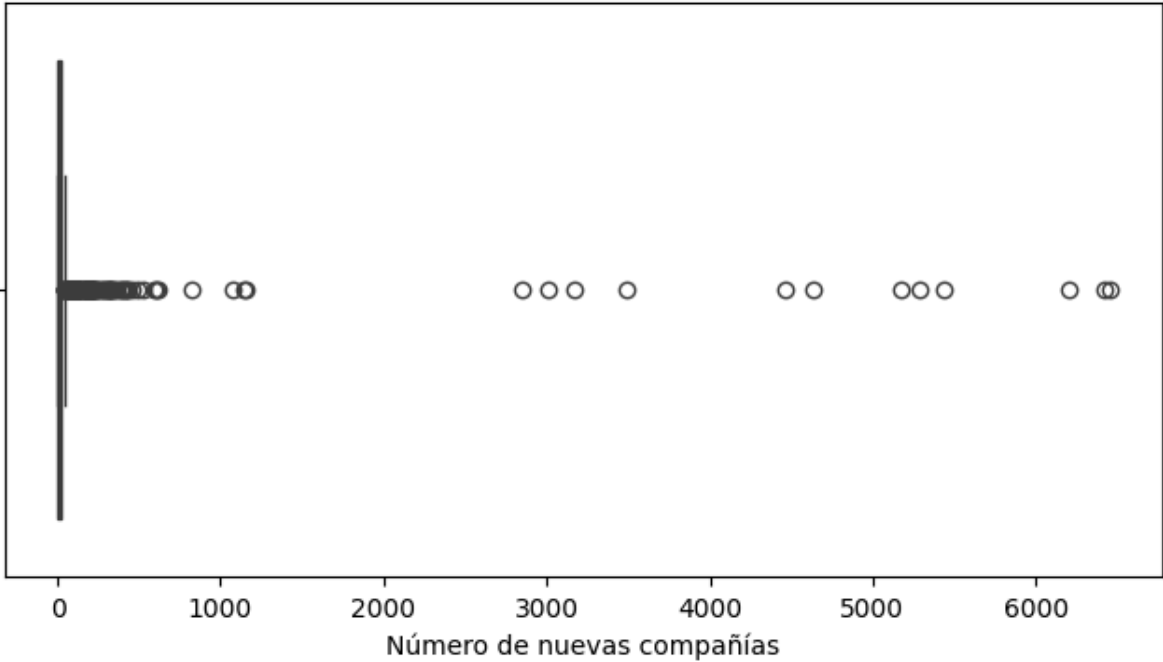
--- Análisis Estadístico: Población ---
Q1: 12592.75 | Mediana: 27025.00 | Q3: 62561.75
Media: 79940.24
Outliers detectados: 115 (8.67% del total)





--- Análisis Estadístico: Número de nuevas compañías ---
Q1: 4.00 | Mediana: 8.00 | Q3: 22.00
Media: 73.50
Outliers detectados: 163 (12.29% del total)

Distribución de Número de nuevas compañías



--- Análisis Estadístico: Valor agregado bruto de producción per cápita ---
Q1: 1788.32 | Mediana: 2467.32 | Q3: 3754.46
Media: 4785.16
Outliers detectados: 165 (12.44% del total)

Distribución de Valor agregado bruto de producción per cápita



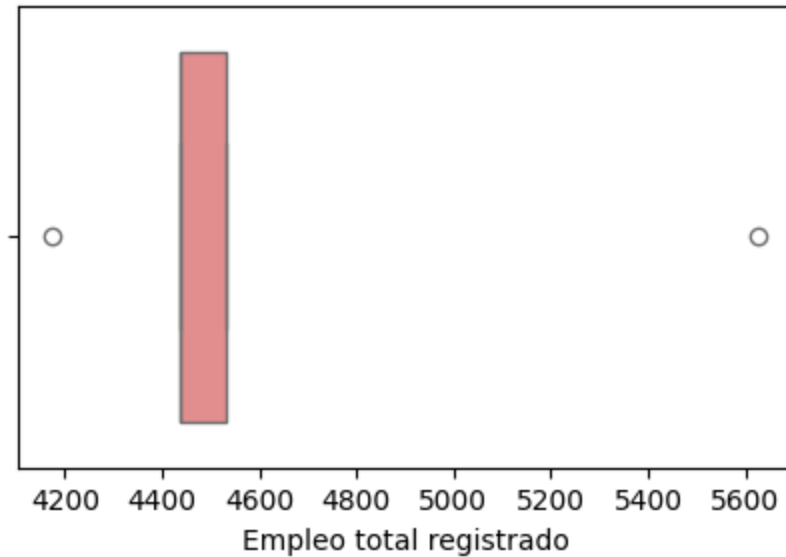
```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3
4 lista_cantones = df_limpio['Cantón'].unique()
5 columna = 'Empleo total registrado'
6
7 print(f"--- Iniciando reporte de cantones con outlier
8
9 for canton in lista_cantones:
10     # Filtrar datos por cantón y periodo
11     df_temp = df_limpio[(df_limpio['Cantón'] == canton
12                          (df_limpio['Año'] >= 2019) &
13                          (df_limpio['Año'] <= 2023)].c
14
15     # Calcular límites
16     Q1 = df_temp[columna].quantile(0.25)
17     Q3 = df_temp[columna].quantile(0.75)
18     IQR = Q3 - Q1
19     limite_inferior = Q1 - 1.5 * IQR
20     limite_superior = Q3 + 1.5 * IQR
21
22     # Identificar outliers
23     outliers = df_temp[(df_temp[columna] < limite_inf
24                          num_outliers = len(outliers)
25
26     # Solo imprimimos y graficamos si encontramos al
27     if num_outliers > 0:
28         print(f"\n[!] Cantón: {canton} | Outliers det
29         # Imprimir detalles de los años y valores
30         print(outliers[['Año', columna]].to_string(in
31
32         # Gráfico
33         plt.figure(figsize=(5, 3))
34         sns.boxplot(x=df_temp[columna], color='lightc
35         plt.title(f'Outliers en: {canton}')
36         plt.show()
37
38     print("\n--- Proceso finalizado ---")
```


--- Iniciando reporte de cantones con outliers en 'Empleo total registrado' ---

[!] Cantón: Antonio Ante | Outliers detectados: 2

Año	Empleo total registrado
2019	5625.17
2021	4175.50

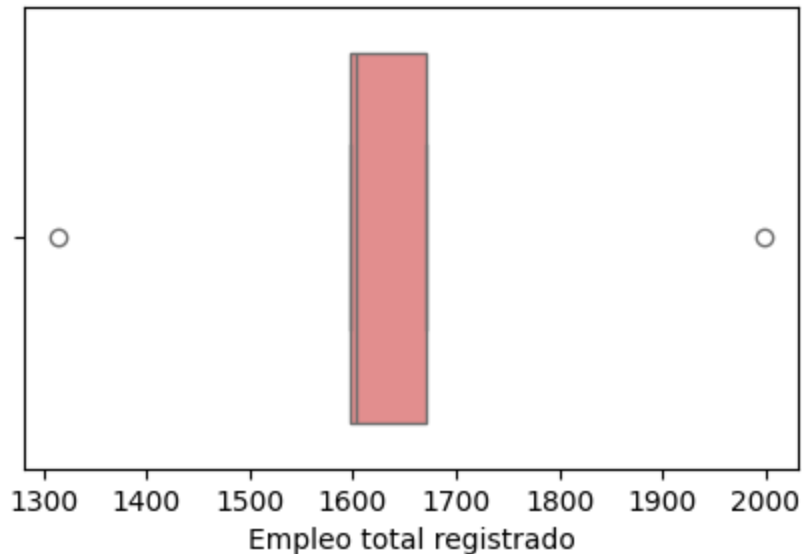
Outliers en: Antonio Ante



[!] Cantón: San Miguel De Urcuquí | Outliers detectados: 2

Año	Empleo total registrado
2019	1997.67
2021	1315.25

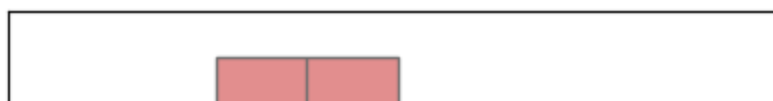
Outliers en: San Miguel De Urcuquí

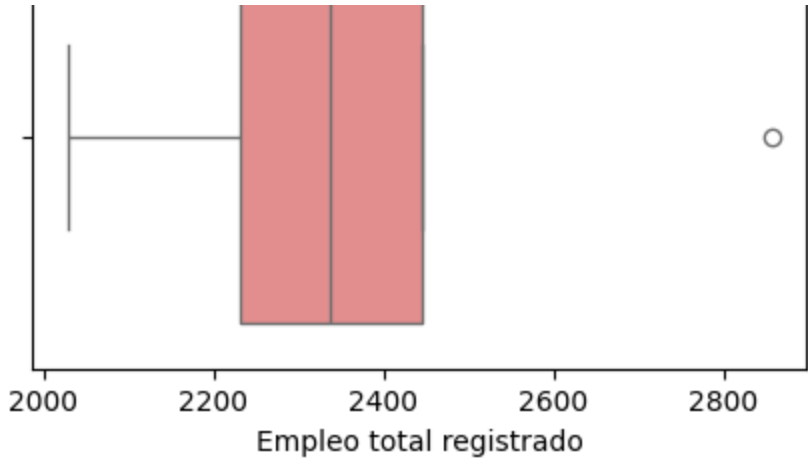


[!] Cantón: Paute | Outliers detectados: 1

Año	Empleo total registrado
2019	2856.08

Outliers en: Paute

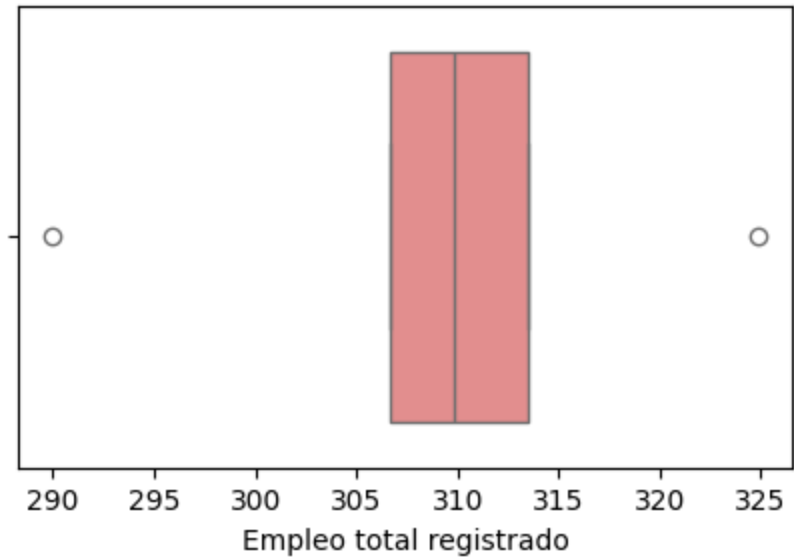




[!] Cantón: Pindal | Outliers detectados: 2

Año	Empleo total registrado
2019	290.00
2023	324.83

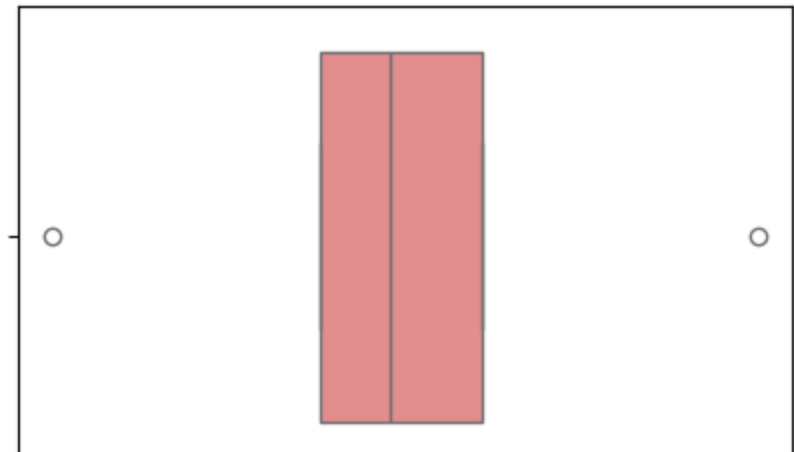
Outliers en: Pindal

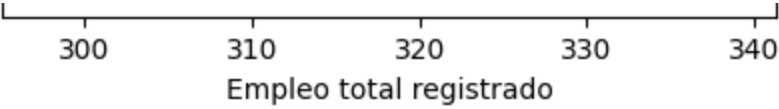


[!] Cantón: Sevilla De Oro | Outliers detectados: 2

Año	Empleo total registrado
2021	339.25
2023	297.17

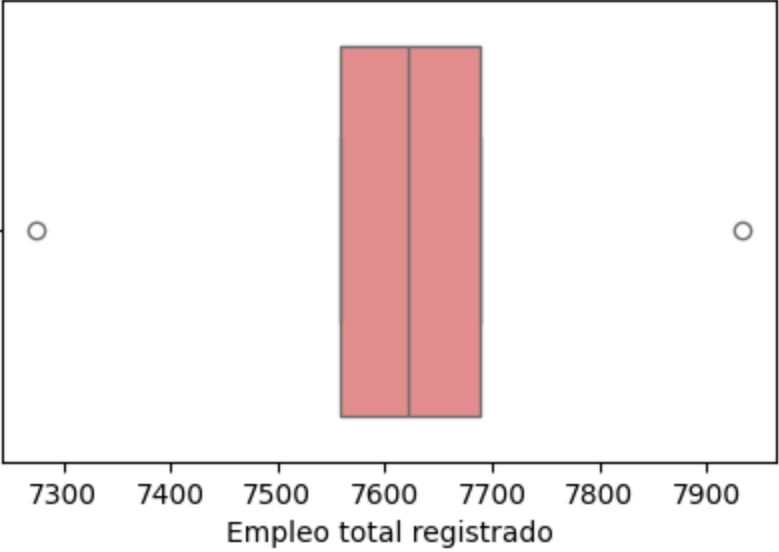
Outliers en: Sevilla De Oro





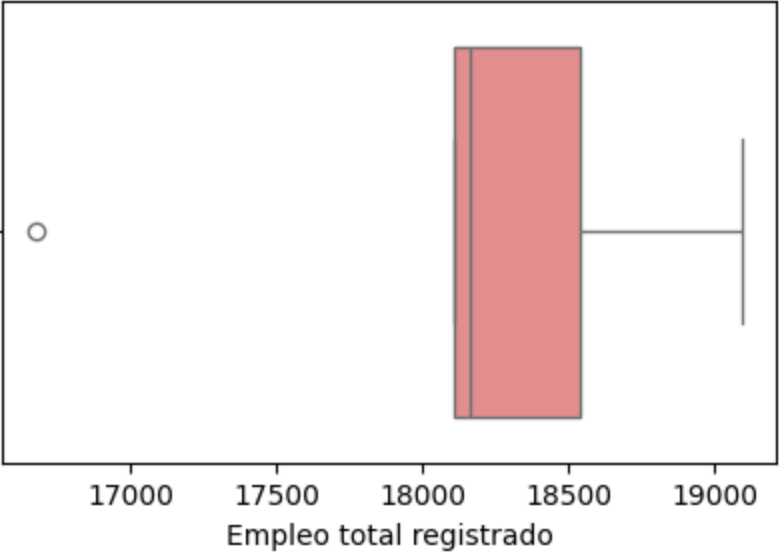
```
[!] Cantón: Camilo Ponce Enríquez | Outliers detectados: 2
Año Empleo total registrado
2020 7275.67
2022 7932.83
```

Outliers en: Camilo Ponce Enríquez



```
[!] Cantón: Babahoyo | Outliers detectados: 1
Año Empleo total registrado
2023 16678.75
```

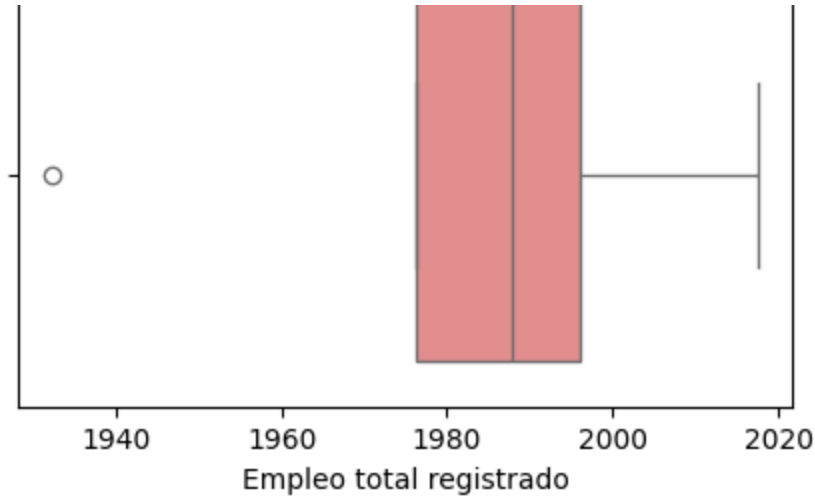
Outliers en: Babahoyo



```
[!] Cantón: Baba | Outliers detectados: 1
Año Empleo total registrado
2019 1932.42
```

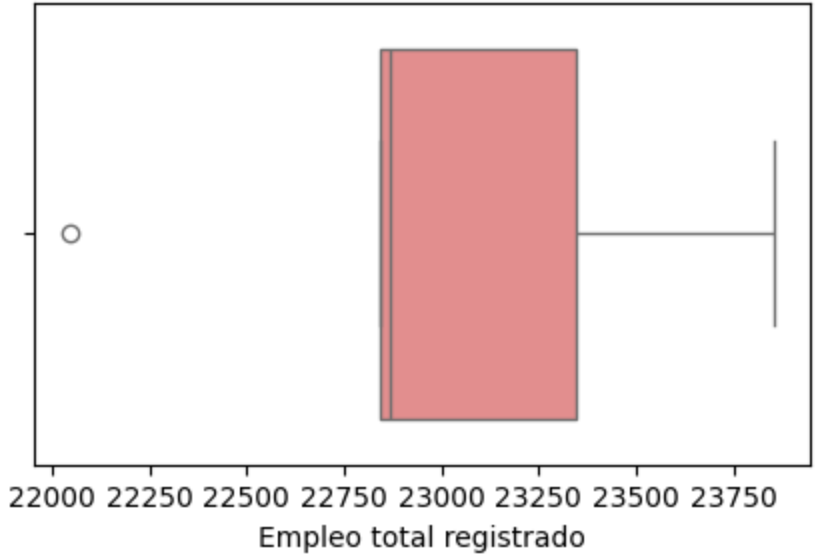
Outliers en: Baba





```
[!] Cantón: Quevedo | Outliers detectados: 1
Año Empleo total registrado
2022 22045.42
```

Outliers en: Quevedo



```
[!] Cantón: Vines | Outliers detectados: 1
Año Empleo total registrado
2021 3590.75
```

Outliers en: Vines

